Technical Document Analyzer

Summary Report and Practical Guide

Built with Python 3 | Standard Libraries Only | Offline Desktop & CLI Utility

1. What This Tool Does

The Technical Document Analyzer is a simple, lightweight program written in Python to help people inspect plain-text technical documents (.txt files). When writing or reviewing documentation, it is often useful to check key numbers—like how many words and paragraphs are in a draft, how many sections were created, and which technical words appear most often.

This tool was built using only Python's standard library. It does not require any third-party packages, has no connection to external web servers, and uses no AI services. Everything runs directly on your computer in a fraction of a second.

2. Project Files and What They Do

The project is split into four simple files so that the code is clean and easy to read:

File Name	Type	Description
main.py	Desktop Application	A clean graphical window built using Tkinter. It lets you browse for a file, view the text in a preview box, see all calculated numbers, and save a report.
cli.py	Command-Line Tool	A terminal version for users who prefer working directly in the command prompt or running scripts without opening a window.
analyzer.py	Core Analysis Engine	Contains the actual calculation functions: reads files safely, cleans punctuation, counts words and paragraphs, and finds frequent words.
test_analyzer.py	Automated Tests	A test script with 7 automated tests to verify that every function works correctly on normal files, empty files, and error cases.

3. How the Calculations Work

• **Word Count:** The program converts text to lowercase so words like *Sensor* and *sensor* match. It replaces punctuation marks (commas, periods, colons, slashes) with spaces so joined words like status:active separate cleanly, then counts the remaining words.

• **Character Count:** Total characters are counted directly. A second count strips spaces, tabs, and newlines to show the exact character volume of the words themselves.

• **Paragraph Count:** Instead of simply counting enter keys, the code looks for transitions from blank lines to text lines. This correctly handles extra blank lines and different operating system line breaks (Windows vs Mac/Linux).

• **Headings:** Detects lines starting with Markdown hashes (# Title, ## Section) and numbered section headers (like 1. Introduction, 1.1 System, or A. Overview).

• **Top Frequent Words:** Filters out numbers and common everyday words (like *the*, *and*, *is*, *of*, *to*). It counts occurrences using a standard Python dictionary and lists the top 10 most frequent words.

4. How the Program Works (Process Flow)

The diagram below outlines the full workflow from when a user selects a file to when the analysis report is saved:

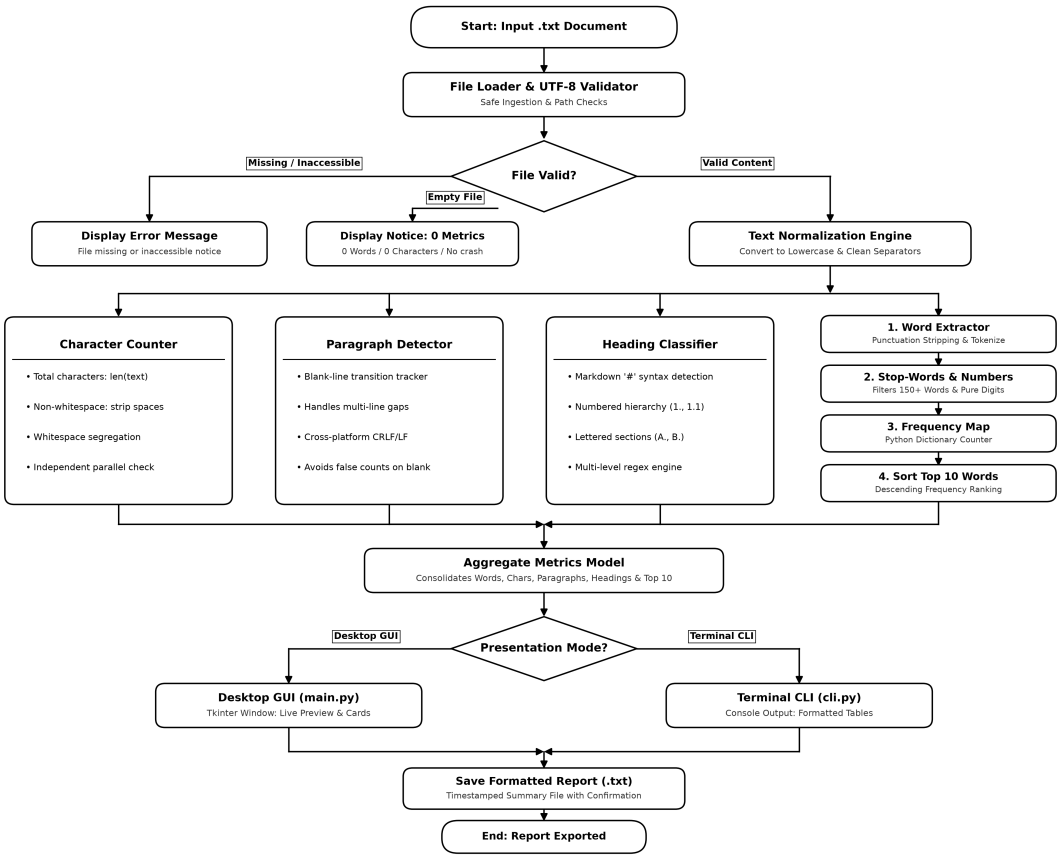


Figure 1: System architecture and execution flow of the Technical Document Analyzer.

Workflow Summary by Stage:

• **Stage 1 (Input & Validation):** The program opens a .txt file and reads it safely using UTF-8 encoding. If the file is missing or inaccessible, an error message is shown. If the file is empty, a notice is displayed and counts show zero.

• **Stage 2 (Text Cleaning & Analysis):** Text is lowercased and stripped of punctuation marks so that capitalization differences do not affect counts. Four independent checks run in parallel: character counting (with and without spaces), paragraph detection (tracking blank lines), heading detection (Markdown # and numbered sections), and word extraction.

• **Stage 3 (Word Frequency & Aggregation):** Common English stop words (like *the*, *and*, *is*) and pure numbers are filtered out. Remaining words are counted in a dictionary and sorted by frequency descending. All metrics are combined into a final results model.

• **Stage 4 (Presentation & Export):** Results are shown in either the Desktop GUI (with document preview pane) or the Terminal CLI. The user can export a clean, formatted .txt analysis report with confirmation before overwriting.

5. Testing and Verification

The project includes an automated test script (test_analyzer.py) that verifies all core functions using Python's built-in unittest framework. All 7 test cases passed in 0.05 seconds:

Test Case	Scenario Tested	What It Checks	Result
Test 1	Normal technical document	Calculates words, characters, paragraphs, headings, and frequent words accurately.	**Passed**
Test 2	Empty file (0 bytes)	Handles zero-length files gracefully without division errors or crashes.	**Passed**
Test 3	Document with only headings	Checks both Markdown '#' headings and numbered '1.', '1.1', 'A.' headings.	**Passed**
Test 4	Mixed case and punctuation	Confirms 'Sensor', 'sensor', and 'SENSOR' count together, stripping symbols.	**Passed**
Test 5	Missing or invalid file path	Catches FileNotFoundError safely and alerts the user.	**Passed**
Test 6	Saving a report to disk	Writes formatted text report to destination and verifies file contents match.	**Passed**
Test 7	Handling a save error	Traps invalid write paths (OSError) without crashing the application.	**Passed**

Results on Sample Technical Documents:

Document	Words	Chars (No Spc)	Paragraphs	Headings	Top Frequent Words
arduino_sensor_system.txt	399	2,469	8	18	sensor (13), pin (8), microcontroller (5), power (5)
software_install_guide.txt	476	2,846	10	20	software (6), installation (5), port (5), system (5)
sample.txt (Packet Analyzer)	393	2,479	7	13	network (13), packet (10), capture (8), protocol (8)

6. Practical Notes and Limitations

To keep the program simple, transparent, and beginner-friendly, here are its intentional boundaries:

- **Plain Text Only:** The program accepts .txt files. It does not parse binary formats like Word (.docx) or PDF.
- **Numbered Steps Counted as Headings:** Plain text has no special heading tags, so any numbered procedural step (such as 1. Connect the cable) looks identical to a section heading and is counted.
- **English Stop Words:** The stop-word list is designed for standard English text. Documents in other languages will require updating the word set.
- **Word Variations:** The frequency counter matches exact spelling after lowercasing. Singular and plural forms (like *sensor* and *sensors*) are counted as separate words.

Project:	Technical Document Analyzer (Python 3)
Interfaces:	Desktop Graphical User Interface (main.py) and Terminal CLI (cli.py)
Test Status:	All 7 automated unit tests passed • Verified on realistic sample files